Lecture 4. Statistical Foundations

Exploratory and confirmatory analysis, samples and populations, sampling distributions, the bootstrap, outliers, models

Contents

4.1 From description to statistical methods **2**

4.2 Samples and populations: a travel policy **2**

4.3 Sampling distributions **3**

4.4 The bootstrap **4**

4.5 Outliers **5**

4.6 Statistical models **6**

4.1 From description to statistical methods

Motivation. Wrangling and visualisation describe the data in hand. To say anything beyond those particular cases, about the next flight or the whole population, we need to know how much the numbers would change with different data. That is the job of statistical methods.

The objective of data science is to extract meaningful information from data. Wrangling makes data easier to interpret and visualisation connects our minds with the data; statistical methods quantify patterns and their strength, find patterns too complex to see, and interpret data, usually through models.

Exploratory data analysis checks data integrity and generates hypotheses ("smoking is related to cancer", "shape and size predict price"), iterating transform–visualise–model. **Confirmatory** analysis checks that a fitted model is appropriate and that a hypothesis holds, either for prediction accuracy or for statistical inference. As a rule, data used for exploration are not used for confirmation.

A statistical method is more than a formula. The t-test for $H_0 : \mu = 0$ rejects when $|t| > 1.96$, $t = \sqrt{n}\,\bar{X}/s$; behind it lie

- a **model**: observations centred at an unknown μ with random error;
- a **fit**: centre and error estimated by $\bar{X}$ and s;
- **assumptions**: numeric, (approximately) normal observations;
- **uncertainty quantification**: the 95% confidence interval $\bar{X} \pm 1.96\, s/\sqrt{n}$;
- **strength of evidence**: the smaller the p-value, the stronger the evidence against the hypothesis.

The key to using such methods correctly is to view the data as a **sample** from a larger **population**, and to imagine the other samples that might have been drawn.

4.2 Samples and populations: a travel policy

Motivation. A concrete decision makes the abstract question precise: how early must a traveller arrive to be late only rarely? Pretending to know the whole population lets us see exactly how much a small sample can mislead.

A business traveller from New York has a meeting in San Francisco at a given time. How much earlier should the scheduled arrival be so that a flight delay makes the traveller late only rarely? Pretend that the 2013 flights to SFO are the whole population:

```
SF <- flights %>% filter(dest == "SFO", !is.na(arr_delay))
set.seed(101)
sf_25 <- SF %>% slice_sample(n = 25)
sf_25 %>% fivenum_tbl(arr_delay)
```

```
## # A tibble: 1 x 8
##       n  mean    sd   min   q25 median   q75   max
##   <int> <dbl> <dbl> <dbl> <dbl>  <dbl> <dbl> <dbl>
## 1    25  0.12  29.8   -38   -23     -5    14   103
```

```
SF %>% fivenum_tbl(arr_delay)
```

```
## # A tibble: 1 x 8
##       n  mean    sd   min   q25 median   q75   max
##   <int> <dbl> <dbl> <dbl> <dbl>  <dbl> <dbl> <dbl>
## 1 13173  2.67  47.7   -86   -23     -8    12  1007
```

`slice_sample()` draws random rows and `set.seed()` makes the pseudo-random draw reproducible. The sample's largest delay is under two hours, but the population contains a delay of 1007 minutes, about 17 hours. Planning for the worst case is absurd; a sensible policy accepts a small risk, say being late 2% of the time, i.e. uses the 98th percentile:

```
sf_25 %>% summarize(q98 = quantile(arr_delay, p = 0.98))
```

```
## # A tibble: 1 x 1
##     q98
##   <dbl>
## 1  67.5
```

```
SF %>% group_by(arr_delay < 90) %>% count() %>% mutate(pct = n / nrow(SF))
```

```
## # A tibble: 2 x 3
## # Groups:   arr_delay < 90 [2]
##   `arr_delay < 90`     n    pct
##   <lgl>            <int>  <dbl>
## 1 FALSE              640 0.0486
## 2 TRUE             12533 0.951
```

```
SF %>% summarize(q98 = quantile(arr_delay, p = 0.98))
```

```
## # A tibble: 1 x 1
##     q98
##   <dbl>
## 1   153
```

A 90-minute policy would fail about 5% of the time, not 2%; the population says about 150 minutes. Computing a percentile is easy; the real question is how good an estimate from a sample is, and how large a sample must be. That is where statistics comes in.

4.3 Sampling distributions

Motivation. A sample statistic is itself random: another sample would give another value. Its sampling distribution, and above all the standard error, measures how reliable it is.

A **statistic** is a number summarising data (mean, standard deviation, median, maximum). Extremes such as the maximum are not useful statistics, because there is no reliable way to tell how well they reflect the population.

How similar would the statistic be on a new sample? With the population at hand we can just draw many samples:

```
n <- 25
num_trials <- 500
sf_25_means <- 1:num_trials %>%
  map_dfr(~ SF %>% slice_sample(n = n) %>% summarize(mean_arr_delay = mean(arr_delay)))
    %>%
  mutate(n = n)
sf_25_means %>% fivenum_tbl(mean_arr_delay)
```

```
## # A tibble: 1 x 8
##       n  mean    sd   min   q25 median   q75   max
##   <int> <dbl> <dbl> <dbl> <dbl>  <dbl> <dbl> <dbl>
## 1   500  3.09  10.2 -20.1 -3.72   1.54     9  56.7
```

Vocabulary:

- the **sample size** n;
- the **sampling distribution**: the collection of the statistic over many trials (the number of trials is not important once large);

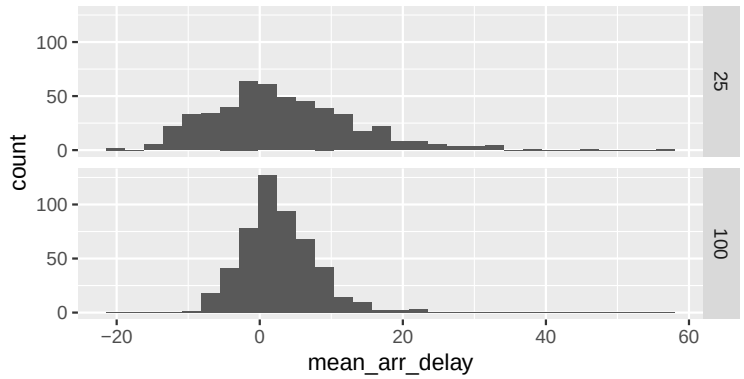
- its **shape**: here somewhat right-skewed;
- the **standard error**: the standard deviation of the sampling distribution, i.e. its width;
- the **95% confidence interval**, computed from the mean and standard error of the sampling distribution, roughly mean ± 2 SE.

For $n = 25$ the sample means range over about 75 minutes, which shows how unreliable such a small sample is. Repeating with $n = 100$:

```
sf_100_means <- 1:num_trials %>%
  map_dfr(~ SF %>% slice_sample(n = 100) %>% summarize(mean_arr_delay = mean(arr_delay))
    ) %>%
  mutate(n = 100)
bind_rows(sf_25_means, sf_100_means) %>%
  group_by(n) %>%
  summarize(mean = mean(mean_arr_delay), se = sd(mean_arr_delay))
```

```
## # A tibble: 2 x 3
##       n  mean    se
##   <dbl> <dbl> <dbl>
## 1    25  3.09 10.2
## 2   100  2.58  4.99
```

```
bind_rows(sf_25_means, sf_100_means) %>%
  ggplot(aes(mean_arr_delay)) + geom_histogram(bins = 30) + facet_grid(n ~ .)
```



Quadrupling n halves the standard error and makes the distribution more symmetric.

The $1/\sqrt{n}$ law [Supplement]

For a sample mean of i.i.d. observations, $\mathrm{Var}(\bar{X}) = \sigma^2/n$, so $\mathrm{SE} = \sigma/\sqrt{n}$: here $\sigma \approx 47.7$ gives 9.5 for $n = 25$ and 4.8 for $n = 100$, matching the simulation. (Sampling without replacement from a finite population multiplies by $\sqrt{(N-n)/(N-1)} \approx 1$ here.) By the central limit theorem the sampling distribution approaches a normal shape as n grows, even for skewed data such as delays.

4.4 The bootstrap

Motivation. In practice we have one sample, not the population, so we cannot draw new samples. The bootstrap pretends that the sample is the population and resamples from it, which approximates the sampling distribution from the data alone.

Resampling draws a new sample from the existing sample. When sampling from a population no case is duplicated; when resampling we sample *with replacement*, and the duplication is what produces variability. Take one sample of $n = 200$ and forget the population:

```
n <- 200
orig_sample <- SF %>% slice_sample(n = n, replace = FALSE)
sf_200_bs <- 1:num_trials %>%
  map_dfr(~ orig_sample %>% slice_sample(n = n, replace = TRUE) %>%
            summarize(mean_arr_delay = mean(arr_delay)))
sf_200_bs %>% fivenum_tbl(mean_arr_delay)
```

```
## # A tibble: 1 x 8
##       n  mean    sd   min   q25 median   q75   max
##   <int> <dbl> <dbl> <dbl> <dbl>  <dbl> <dbl> <dbl>
## 1   500  8.58  4.16 -1.94  5.47   8.42  11.3  23.7
```

```
sf_200_pop <- 1:num_trials %>%
  map_dfr(~ SF %>% slice_sample(n = n) %>% summarize(mean_arr_delay = mean(arr_delay)))
sf_200_pop %>% fivenum_tbl(mean_arr_delay)
```

```
## # A tibble: 1 x 8
##       n  mean    sd   min   q25 median   q75   max
##   <int> <dbl> <dbl> <dbl> <dbl>  <dbl> <dbl> <dbl>
## 1   500  2.69  3.32 -6.41 0.391   2.65  4.86  18.0
```

The **bootstrap distribution** (first table) is centred at the original sample's mean rather than the population mean, but its standard deviation approximates the true standard error (second table). For moderate to large samples and enough resamples it has been proven to approximate the aspects of the sampling distribution we care about most, such as standard errors and quantiles (Efron and Tibshirani, 1993).

Back to the travel policy: the 98th percentile from the sample, and its bootstrap reliability:

```
orig_sample %>% summarize(q98 = quantile(arr_delay, p = 0.98))
```

```
## # A tibble: 1 x 1
##     q98
##   <dbl>
## 1  209.
```

```
sf_200_bs_q98 <- 1:num_trials %>%
  map_dfr(~ orig_sample %>% slice_sample(n = n, replace = TRUE) %>%
            summarize(q98 = quantile(arr_delay, p = 0.98)))
sf_200_bs_q98 %>% fivenum_tbl(q98)
```

```
## # A tibble: 1 x 8
##       n  mean    sd   min   q25 median   q75   max
##   <int> <dbl> <dbl> <dbl> <dbl>  <dbl> <dbl> <dbl>
## 1   500  201.  58.2  74.1  152.    208  251.   360
```

The standard error of the estimated 98th percentile is large relative to the estimate: extreme quantiles need much more data than means.

4.5 Outliers

Motivation. Rare extreme events are exactly what a travel policy must anticipate. Looking at them can suggest which conditions make long delays likely.

Consider delays of at least 7 hours (420 minutes) as extreme:

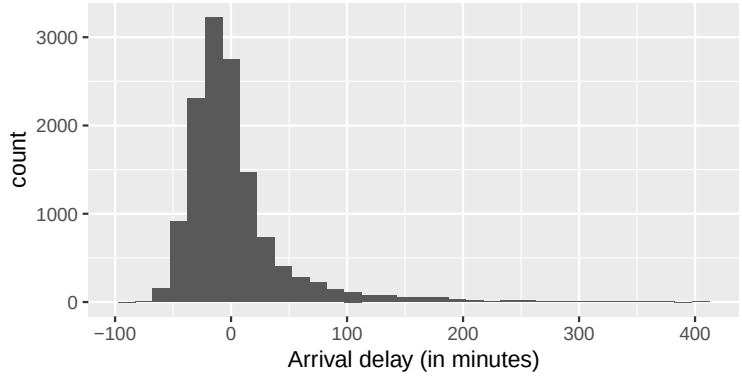
```
SF %>% filter(arr_delay >= 420) %>% select(month, day, dep_delay, arr_delay, carrier)
```

```
## # A tibble: 7 x 5
##   month   day dep_delay arr_delay carrier
```

```
##   <int> <int>     <dbl>     <dbl> <chr>
## 1    12     7       374       422 UA
## 2     7     6       589       561 DL
## 3     7     7       629       676 VX
## 4     7     7       653       632 VX
## 5     7    10       453       445 B6
## 6     7    10       432       433 VX
## # i 1 more row
```

Removing them shows the main part of the distribution: most flights arrive on time or less than an hour late.

```
SF %>% filter(arr_delay < 420) %>%
  ggplot(aes(arr_delay)) + geom_histogram(binwidth = 15) +
  labs(x = "Arrival delay (in minutes)")
```



The outliers suggest that month or carrier matter. Counting long delays (over an hour) by month and by carrier:

```
SF %>% mutate(long_delay = arr_delay > 60) %>%
  count(month, long_delay) %>%
  pivot_wider(names_from = month, values_from = n)
```

```
## # A tibble: 2 x 13
##   long_delay   `1`   `2`   `3`   `4`   `5`   `6`   `7`   `8`   `9`  `10`  `11`  `12`
##   <lgl>      <int> <int> <int> <int> <int> <int> <int> <int> <int> <int> <int> <int>
## 1 FALSE        856   741   812   993  1128   980   966  1159  1124  1177  1107  1093
## 2 TRUE          29    21    61   112    65   209   226    96    65    36    51    66
```

```
SF %>% mutate(long_delay = arr_delay > 60) %>%
  count(carrier, long_delay) %>%
  pivot_wider(names_from = carrier, values_from = n)
```

```
## # A tibble: 2 x 6
##   long_delay    AA    B6    DL    UA    VX
##   <lgl>      <int> <int> <int> <int> <int>
## 1 FALSE       1250   934  1757  6236  1959
## 2 TRUE         148    86    91   492   220
```

June and July are problem months, and Delta (DL) performs reasonably. This hints at a policy: arrive extra early in June and July, and consider Delta for travel to SFO.

4.6 Statistical models

Motivation. Conditioning on month and carrier explained part of the variation in delays. A statistical model does this systematically, relating the response to several variables at once and separating signal from noise.

"Essentially, all models are wrong, but some are useful" (G. E. P. Box). A model is an idealised representation of a system, like a weather forecast, which is useful though often wrong. We build models

1. to **understand** complex phenomena (which factors matter?), preferring simple interpretable models; and
2. to **predict** unseen data accurately, sometimes with uninterpretable black-box models.

Most of the time we balance interpretability and accuracy.

With response Y and predictors $X = (X_1, \ldots, X_p)$, we assume

$$Y = f(X) + \epsilon,$$

with f fixed but unknown and ϵ random error with mean zero. We estimate f for **prediction**, $\hat{Y} = \hat{f}(X)$, where accuracy matters, or for **inference**, where the form of f matters: which predictors are associated with the response, how, and whether a linear form suffices.

Proposition 4.1 (reducible and irreducible error)

If $\hat{f}$ and X are fixed and $\mathbb{E}\epsilon = 0$, then

$$\mathbb{E}[(Y - \hat{Y})^2] = [f(X) - \hat{f}(X)]^2 + \mathrm{Var}(\epsilon).$$

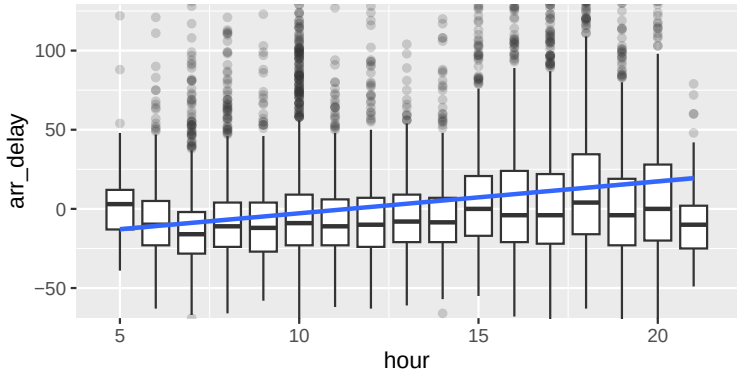
Proof [Supplement]. $Y - \hat{Y} = \{f(X) - \hat{f}(X)\} + \epsilon$, where the first term is a constant c. Hence $\mathbb{E}(Y - \hat{Y})^2 = c^2 + 2c\,\mathbb{E}\epsilon + \mathbb{E}\epsilon^2 = c^2 + \mathrm{Var}(\epsilon)$. ■

The first term, the **reducible error**, can be lowered by a better estimate $\hat{f}$; the second, the **irreducible error**, cannot. Methods aim to minimise the reducible error.

Parametric models specify $f(X;\theta)$ with finitely many parameters, e.g. $Y \approx \beta_0 + \beta_1 X_1 + \cdots + \beta_p X_p$; **nonparametric** models such as k-nearest neighbours make no explicit assumption about the form of f. More flexible methods are generally less interpretable.

Example. Does the scheduled hour of departure affect delays, as the theory that delays cascade through the day predicts? Flights to SFO are scheduled from 5 am to 9 pm. A linear model tracks the mean delay over the day:

```
SF %>% ggplot(aes(hour, arr_delay)) +
  geom_boxplot(aes(group = hour), outlier.alpha = 0.2) +
  geom_smooth(method = "lm", formula = y ~ x) +
  coord_cartesian(ylim = c(-60, 120))
```



```
mod1 <- lm(arr_delay ~ hour, data = SF)
broom::tidy(mod1)
```

```
## # A tibble: 2 x 5
##   term        estimate std.error statistic   p.value
##   <chr>          <dbl>     <dbl>     <dbl>     <dbl>
## 1 (Intercept)   -22.9     1.23       -18.6 2.88e- 76
## 2 hour            2.01    0.0915      22.0 1.78e-105
```

The mean arrival delay increases by about 2 minutes for each hour later in the day, and the tiny p-value says such a slope would be very unlikely if there were no relationship.